Міністерство освіти і науки України
Національний технічний університет «Дніпровська політехніка»

ЗВІТ

про виконання практичної роботи №1

з дисципліни
«Поглиблене програмування в середовищі Java»

Тема роботи:
“Системи контролю версій (VCS). GIT”

Виконала студентка: гр.122-22-4

Мауріна Катерина Віталіївна

Перевірив: доцент

Мінєєв Олександр Сергійович

Дніпро
2026

Мета завдання: розробити повноцінний клієнт-серверний застосунок на мові Java, що реалізує обрану бізнес-логіку (наприклад, прототип вашої майбутньої дипломної роботи або сервіс для автоматизації власних задач).

Хід роботи:

1. Ініціалізація та налаштування бекенд-середовища: розгорнути базовий застосунок на стеку Java 17+ та Spring Boot 3.x і налаштувати підключення до реляційної бази даних (H2/PostgreSQL/MySQL) із використанням Spring Data JPA.

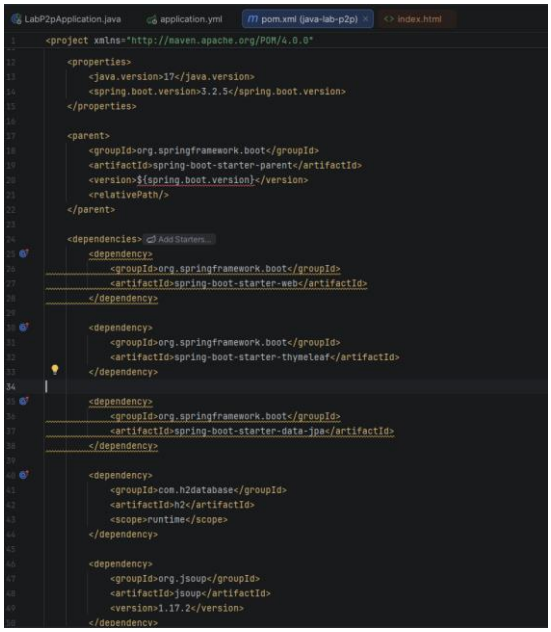

Рис.1.1. Залежності та версії проєкту у pom.xml

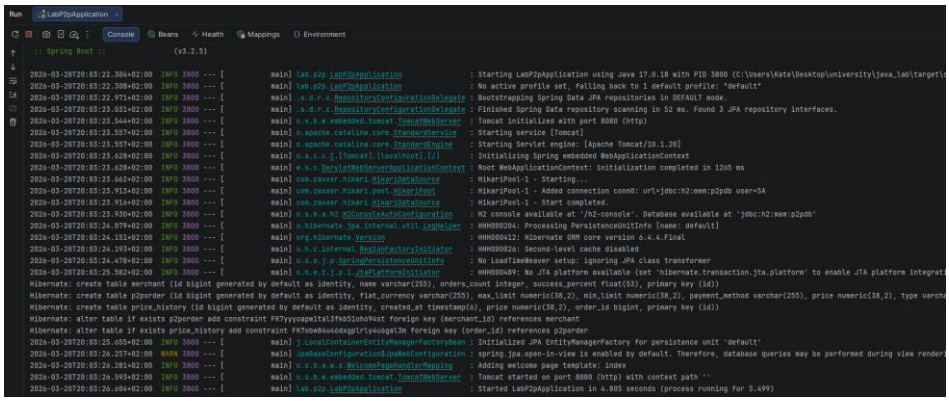

Рис.1.2. Успішний запуск Spring Boot і підключення до БД

Створено проєкт на Java 17 та Spring Boot 3.x, додано залежності для веб-шару, Spring Data JPA та драйвера обраної СУБД. У конфігурації задано

параметри підключення до БД. Це дозволяє запускати застосунок як єдине середовище для роботи з API та збереженням даних у реляційній базі.

2. Проєктування моделі даних: спроєктувати та реалізувати схему БД, що містить щонайменше 2-3 взаємопов’язані сутності з відношеннями One-to-Many або Many-to-Many.

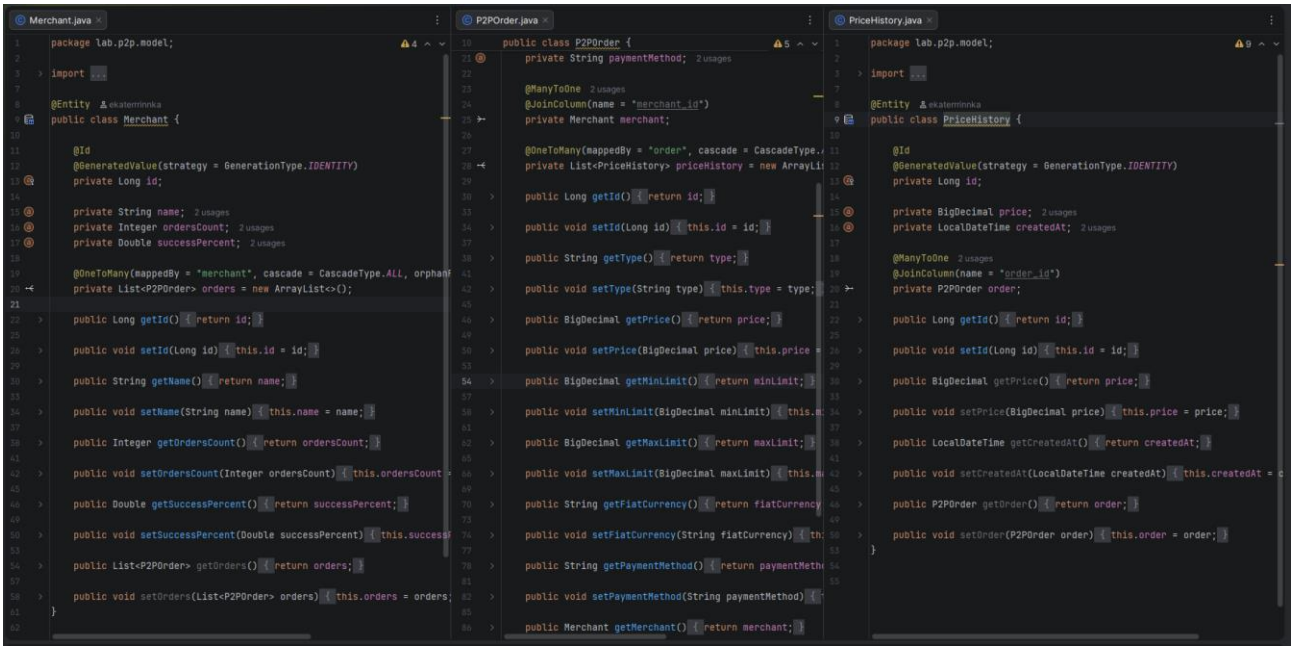

Рисунок 2.1 – Модель даних і зв’язки між сутностями

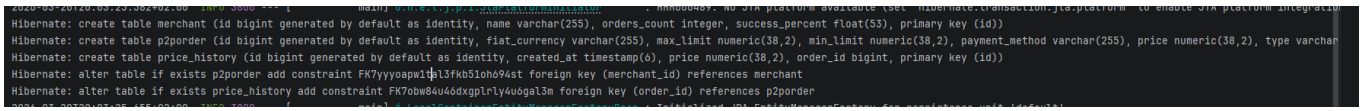

Рисунок 2.2 – Автоматичне створення таблиць і зовнішніх ключів

Спроєктовано доменну модель із мінімум 2–3 пов’язаними сутностями; між ними задано відношення один-до-багатьох. Зв’язки реалізовано через анотації JPA на рівні сутностей, що відповідає логі предметної області (наприклад, мерчант → ордери → історія цін).

3. Інтеграція та обробка зовнішніх даних: створити модуль імпорту (парсер або API-клієнт) для автоматизованого отримання даних із зовнішнього джерела та їх обробки у форматі JSON або HTML (через Jackson/Gson або Jsoup).

```java
ExternalP2PClient.java
public class ExternalP2PClient {
    public List<ExternalP2POfferDto> fetchOffers() {  1 usage  ekaterrrinnka

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(sourceUrl))
                .timeout(Duration.ofSeconds(15))
                .header( name: "Content-Type",  value: "application/json")
                .header( name: "Accept",  value: "application/json")
                .header( name: "Accept-Encoding",  value: "gzip")
                .header( name: "User-Agent",  value: "Mozilla/5.0")
                .header( name: "Origin",  value: "https://p2p.binance.com")
                .header( name: "Referer",  value: "https://p2p.binance.com/")
                .POST(HttpRequest.BodyPublishers.ofString(requestBody))
                .build();

        try {
            HttpResponse<byte[]> response = httpClient.send(request, HttpResponse.BodyHandlers.ofByteArray());
            if (response.statusCode() < 200 || response.statusCode() >= 300) {
                return list;
            }

            String responseBody = decodeResponseBody(response);
            JsonNode root = objectMapper.readTree(responseBody);
            JsonNode data = root.path( s: "data");
            if (!data.isArray()) {
                return list;
            }

            for (JsonNode item : data) {
                JsonNode adv = item.path( s: "adv");
                JsonNode advertiser = item.path( s: "advertiser");

                ExternalP2POfferDto dto = new ExternalP2POfferDto();
                dto.setType(tradeType);
                dto.setPrice(readDecimal(adv,  field: "price"));
                dto.setMinLimit(readDecimal(adv,  field: "minSingleTransAmount"));
                dto.setMaxLimit(readDecimal(adv,  field: "dynamicMaxSingleTransAmount"));
                dto.setFiatCurrency(adv.path( s: "fiatUnit").asText(fiat));
                dto.setPaymentMethod(readPaymentMethod(adv.path( s: "tradeMethods")));
```

Рисунок 3.1 – API-клієнт для отримання зовнішніх даних

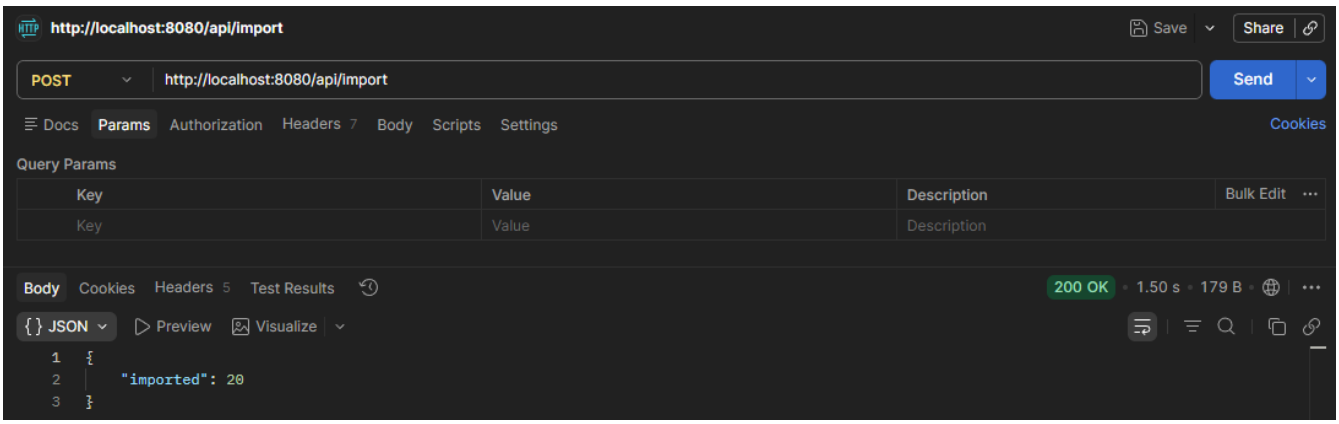

Рис.3.2. Виконання імпорту через endpoint /api/import

Реалізовано модуль отримання даних із зовнішнього джерела через HTTP-клієнт; відповідь у форматі JSON обробляється засобами парсингу і перетворюється на внутрішні структури даних. Для зручності автоматизації передбачено endpoint імпорту, який ініціює завантаження та подальше збереження/оновлення записів у системі.

4. Реалізація REST API та бізнес-логіки: реалізувати контролери з повною підтримкою CRUD-операцій для основних сутностей, використовуючи DTO для передачі даних між шарами застосунку.

```java
P2POrderController.java

package lab.p2p.controller;

import ...

@RestController
@RequestMapping("/api/orders")
public class P2POrderController {

    private final P2POrderService orderService;
    private final MerchantRepo merchantRepo;

    public P2POrderController(P2POrderService orderService, MerchantRepo merchantRepo) {
        this.orderService = orderService;
        this.merchantRepo = merchantRepo;
    }

    @GetMapping
    public List<P2PAdDTO> getAll() {...}

    @GetMapping("/{id}")
    public ResponseEntity<P2PAdDTO> getById(@PathVariable Long id) {...}

    @PostMapping
    public ResponseEntity<P2PAdDTO> create(@RequestBody P2PAdDTO dto) {...}

    @PutMapping("/{id}")
    public ResponseEntity<P2PAdDTO> update(@PathVariable Long id, @RequestBody P2PAdDTO dto) {...}

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        orderService.deleteById(id);
        return ResponseEntity.noContent().build();
    }

    @GetMapping("/filter")
    public List<P2PAdDTO> filter(@RequestParam(name = "minLimit", required = false) BigDecimal minLimit) {
        return orderService.filter(minLimit)
                .stream()
                .map(this::toDto)
```

Рис.4.1. CRUD-операції REST API для сутності ордера

```java
P2PAdDTO.java

package lab.p2p.dto;

import java.math.BigDecimal;

public class P2PAdDTO {

    private Long id;
    private String type;
    private BigDecimal price;
    private BigDecimal minLimit;
    private BigDecimal maxLimit;
    private String fiatCurrency;
    private String paymentMethod;

    private Long merchantId;
    private String merchantName;
    private Integer merchantOrdersCount;
    private Double merchantSuccessPercent;

    public Long getId() { return id; }

    public void setId(Long id) { this.id = id; }

    public String getType() { return type; }

    public void setType(String type) { this.type = type; }

    public BigDecimal getPrice() { return price; }

    public void setPrice(BigDecimal price) { this.price = price; }

    public BigDecimal getMinLimit() { return minLimit; }

    public void setMinLimit(BigDecimal minLimit) { this.minLimit = minLimit; }

    public BigDecimal getMaxLimit() { return maxLimit; }

    public void setMaxLimit(BigDecimal maxLimit) { this.maxLimit = maxLimit; }
```

```java
P2POrderController.java

package lab.p2p.controller;

import ...

@RestController
@RequestMapping("/api/orders")
public class P2POrderController {

    private final P2POrderService orderService;
    private final MerchantRepo merchantRepo;

    public P2POrderController(P2POrderService orderService, MerchantRepo merchantRepo) {
        this.orderService = orderService;
        this.merchantRepo = merchantRepo;
    }

    @GetMapping
    public List<P2PAdDTO> getAll() {...}

    @GetMapping("/{id}")
    public ResponseEntity<P2PAdDTO> getById(@PathVariable Long id) {...}

    @PostMapping
    public ResponseEntity<P2PAdDTO> create(@RequestBody P2PAdDTO dto) {...}

    @PutMapping("/{id}")
    public ResponseEntity<P2PAdDTO> update(@PathVariable Long id, @RequestBody P2PAdDTO dto) {...}

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        orderService.deleteById(id);
        return ResponseEntity.noContent().build();
    }

    @GetMapping("/filter")
    public List<P2PAdDTO> filter(@RequestParam(name = "minLimit", required = false) BigDecimal minLimit)

    @GetMapping("/spread/mono-abank")
    public SpreadDTO getMonoAbankSpread(
```

Рис.4.2. Використання DTO для передачі даних

Для основної сутності реалізовано повний набір CRUD через REST-контролер: отримання списку та одного запису, створення, оновлення та видалення. Для відокремлення внутрішньої моделі від контракту API використано DTO: дані з клієнта приймаються через @RequestBody, відповіді формуються у вигляді DTO або ResponseEntity.

5. Серверна візуалізація інтерфейсу: реалізувати серверний UI-шар (наприклад, на Thymeleaf) для зручного відображення зібраних і оброблених даних.

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8"/>
    <meta name="viewport" content="width=device-width, initial-scale=1"/>
    <title>P2P Monitor</title>
    <link rel="preconnect" href="https://fonts.googleapis.com"/>
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin/>
    <link href="https://fonts.googleapis.com/css2?family=Outfit:wght@400;500;600;700&display=swap" rel="stylesheet"/>
    <link rel="stylesheet" th:href="@{/css/dashboard.css}"/>
</head>
<body>
<div class="page-wrap">
    <header class="hero">
        <h1>P2P дашборд</h1>
        <p>Моніторинг спреду та ордерів у реальному часі</p>
    </header>

    <div class="card">
        <h2>Спред Mono / Abank (UAH)</h2>
        <p class="spread-lines" th:if="${spread != null}">
            Mono: <b class="mono accent" th:text="${spread.monoPrice}">-</b>,
            Abank: <b class="mono accent" th:text="${spread.abankPrice}">-</b><br/>
            Спред: <b class="mono" th:text="${spread.spread}">-</b>
            (<span class="mono accent" th:text="${spread.spreadPercent}">-</span>%)
        </p>
        <p class="empty-hint" th:if="${spread == null}">Немає даних для спреду.</p>
    </div>

    <div class="card">
        <h2>P2P ордери</h2>
        <div class="table-scroll">
            <table>
                <thead>
                <tr>
                    <th>ID</th>
                    <th>Тип</th>
                    <th>Ціна</th>
                    <th>Мін. ліміт</th>
                    <th>Макс. ліміт</th>
                    <th>Фіат</th>
```

Рис.5.1. Thymeleaf-шаблон сторінки дашборду

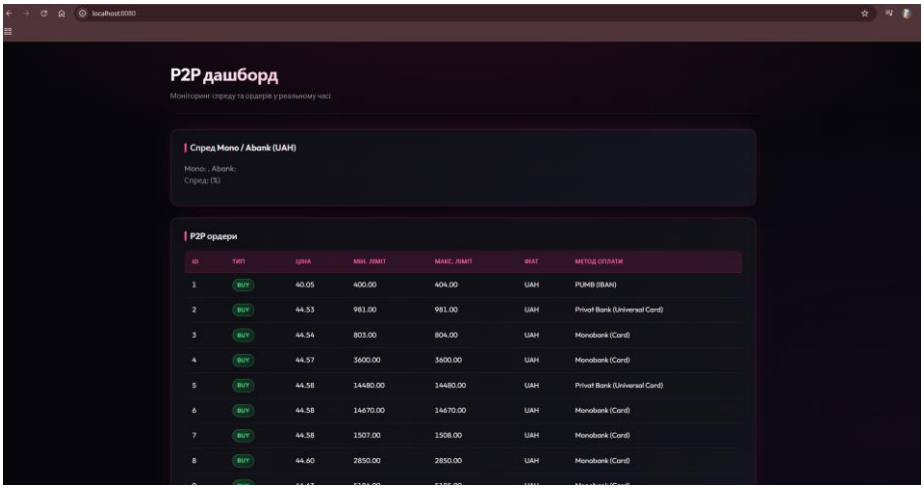

Рис.5.2. Відображення даних у веб-інтерфейсі

Додано серверний UI на Thymeleaf: HTML-шаблон отримує дані з контролера та динамічно відображає їх за допомогою директив (th:text, th:each та інших). Це забезпечує зручний перегляд зібраних і оброблених даних у браузері без окремого SPA-клієнта.

Висновок: освоїла повний базовий цикл розробки бекенд-застосунку на Java 17 + Spring Boot: від налаштування проєкту та БД до побудови моделі даних, інтеграції із зовнішнім API, реалізації CRUD REST API і візуалізації даних

через Thymeleaf. На практиці закріпила роботу з JPA-зв’язками, DTO, обробкою JSON та тестуванням endpoint-ів у Postman. У результаті отримано працездатний застосунок, який автоматично імпортує дані, зберігає їх у БД та відображає у веб-інтерфейсі.